Intelligent Carpool Routing for Urban Ridesharing by Mining GPS Trajectories

Wen He, Kai Hwang, *Life Fellow, IEEE*, and Deyi Li

***Abstract*—To support an efficient carpooling service in heavy urban traffic, we propose an intelligent routing scheme based on mining Global Position System trajectories from shared riders. The carpooling system provides many-to-many services with multiple pickup and dropping points. To join a daily carpooling group, the riders must accept a compromised route that is efficient after merging the routes that are preferred by all qualified riders. We developed three frequency-correlated algorithms for route mining, rider selection, and route merging in an urban carpool service. Our approach can cope with the traffic dynamics to yield a suboptimal shared route. Our scheme was successfully tested under heavy Beijing traffic over hundreds of riders. We developed performance metrics to measure the service cost and mileage saved. The ultimate goal is to minimize the riding distances and the transportation costs, and thus alleviate urban traffic jams.**

***Index Terms*—Carpooling service, Global Positioning System (GPS) trajectory mining, intelligent route merging, urban traffic management.**

I. Introduction

TRAFFIC congestion has become a serious problem in large metropolitan areas. Annoying traffic jams not only degrade quality of life but also negatively impact productivity, the economy, and the environment. To alleviate this problem, one can provide optimal routing through carpool practices [1]. We use a GPS-assisted mining approach to group riders with similar preferred routes to achieve efficient ridesharing to reduce traffic jams, save energy, and reduce transport costs [9].

By mining the GPS-assisted trajectories, we can work out some semioptimal solutions under certain QoS constraints. This trajectory information can greatly help riders build sufficient confidence in joining a carpool group. Frequent routes are rather personalized [4], [5]. The selected carpool route could be dynamically updated to cope with the changing traffic conditions.

This paper is significantly expanded from our earlier workshop paper [2], which was presented at the Association for Computing Machinery (ACM) Special Interest Group on Knowledge Discovery and Data Mining (SIGKDD) International Workshop on Urban Computing in 2012. All the text in the body, algorithms, theorems, and performance plots are newly generated in this journal version.

Based on the driver property and the vehicle use, three carpool models were suggested in the past [6]. Our model covers both company-supported and private carpool services within large metropolitan areas. The vehicle used sets a limit on the carpool size. Riders share their frequent routes to generate a commonly accepted route. The frequent route is generated under different transport modes, including a private car, a taxi, a bus, the subway, or simply walking. The routing optimization goal is to minimize the driving distance, reduce commute costs, protect the environment, and alleviate urban traffic problems.

The rest of this paper is organized as follows. Section II reviews related works and distinguishes our approach from previous schemes. Section III formulates the model in solving the carpooling problem. Section IV presents the frequent routes' mining algorithm. Section V specifies the carpool recommendation strategies. Section VI gives the shared route generation algorithm. We evaluate the carpool routing scheme in Section VII. We report the GPS-assisted routing experiments and the performance results measured. Finally, we summarize our contributions and suggest further research work.

II. Related Work and Our Approach

Surveys of the dynamic carpool problem can be found in [3] and [12]–[14]. In [15], a more flexible collaborative program for carpooling is based on riders' availability and preferences. In [8], an approach was proposed to provide incentive for private drivers to engage in ridesharing. Yan *et al.* [16] proposed a model for long-term many-to-many carpooling services.

Trasarti *et al.* [17] mined the user profiles from digital traces for carpooling. Coltin and Veloso [18] tried to reduce the total drive mileage by transferring passengers between multiple vehicles. Chen *et al.* [5] proposed a method to mine personal routes from users' trajectory data. Chang *et al.* [4]

Manuscript received August 2, 2013; revised October 3, 2013, December 26, 2013, and February 16, 2014; accepted March 19, 2014. Date of publication May 8, 2014; date of current version September 26, 2014. This work was supported in part by the Chinese Basic Research (973) Program under Grant 2011CB302505; by the National Science Foundation of China under Grants 61034005, 91120306, and 61273213; and by Guangdong Innovation Research and Development Team under Grant 201001D0104726115. The work of K. Hwang was supported in part by the EMC Corporation and in part by Intellectual Ventures, LLC. The Associate Editor for this paper was Q. Kong.

W. He is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China and also with the Institute of Chinese Electrical System Engineering, Beijing 100840, China (e-mail: he-w09@mails.tsinghua.edu.cn).

K. Hwang is with the Department of Electrical Engineering, University of Southern California, Los Angeles, CA 90089, USA (e-mail: kaihwang@usc.edu).

D. Li is with the School of Software Engineering, Tsinghua University, Beijing 100084, China (e-mail: lidy@cae.cn).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TITS.2014.2315521

1524-9050 © 2014 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.

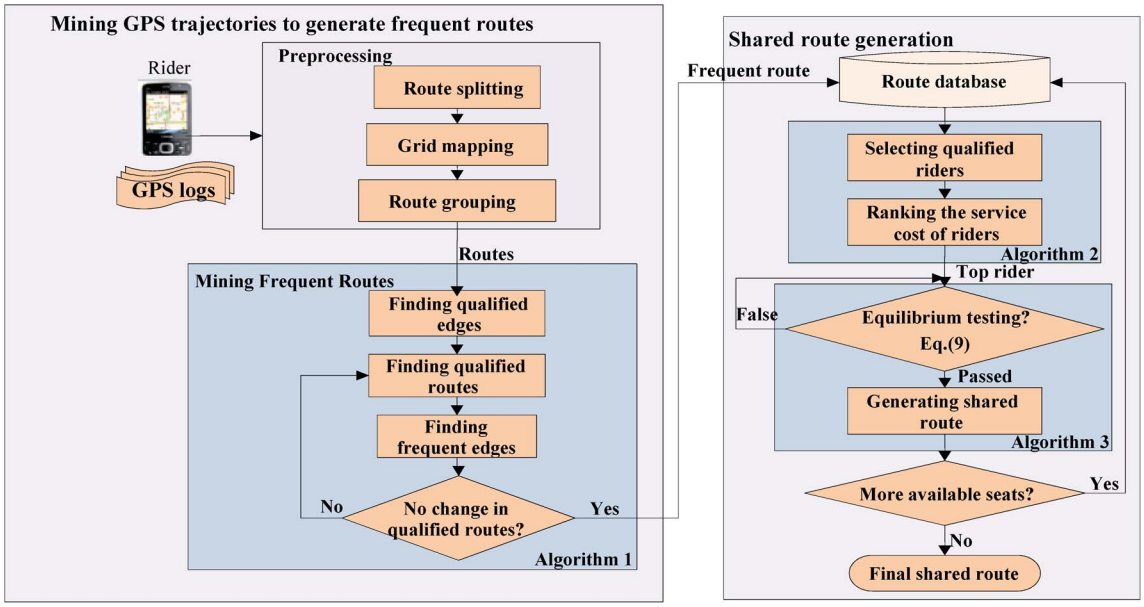


Fig. 1. Carpooling model for generating a suboptimal ridesharing route under heavy urban traffic.

discovered the top-k personalized routes from a trained road network. Giannotti *et al.* [19] projected all trajectories into a geographical grid using grid density to select the regions of interest. We mine similar routes qualified from candidate riders at all times.

In [10], a travel sequence recommendation system is proposed. Lee *et al.* [20] proposed to use a frequent sequential pattern for trajectory classification. Roh *et al.* [21] proposed a system for querying patterns in trajectory. Traffic condition estimation was studied in [11], [22], and [23]. A vehicle trajectory was analyzed in [24] and [25]. Route recommendation was done by Sheng *et al.* [26].

We propose an incremental carpooling model, which differs from previous approaches mainly in the mining algorithm and the optimal route merging process. Our incremental model is very different from the models used in [6], [14], and [16]. Our model is specifically designed to cope with dynamic rider changes, whereas a static carpool must be completely regrouped with the departure or replacement of some shared riders.

A static carpool model could be considered a special case of the dynamic model. However, the two models cannot be easily switched due to different carpooling policies and procedures applied. Our carpool route is pairwise merged from a rider's frequent routes. The carpooling group starts with two riders until all seats are taken. When the car is not fully seated, additional riders are retrieved from the trajectory database. Each new rider may affect the shared route that is to be updated dynamically.

We use an equilibrium test to decide whether the new rider can be inserted properly so that all riders can accept the adjusted route when the carpool group varies in size. Many to many refers to the fact that multiple riders with different (but close-by) pickup points and destinations can share the carpool service. Our carpooling model applies to ridesharing using either a private car or a company-provided vehicle. We support dynamic join and leave by shared riders.

III. GPS Trajectory and Routing Model

In this paper, we focus on recommending riders to share vehicles based on the riders' daily frequent routes. We mine frequent routes from a rider's trajectory logs. There are two rider roles in this carpool system, i.e., a driver who owns a car and could offer seats to other riders and a passenger asking for a ride.

A. GPS-Assisted Carpooling Model

An optimized carpool routing model is developed, taking into account riders' preferences and the dynamic join-and-leave policy. To begin, a vehicle and a driver are assumed to be available. Qualified riders are recruited to join the carpool group one after another until all passenger seats are taken. Departing riders can be replaced by newly qualified riders.

Such a carpool group requires working out an optimized ridesharing route (or a series of routes) that can satisfy all existing riders on a daily basis. The carpooling route is dynamically adjusted with every rider departing or joining. By default, we start with two riders, including a driver, and add more until the vehicle is full. Thus, the driver must cover all passengers on their routes during the merging process.

Fig. 1 shows the architecture of such a proactive carpooling system. There are two major subsystems, i.e., *trajectory mining* and *carpool routing*. The first subsystem processes each user's trajectory log recorded at a rider's GPS device. The second subsystem runs on the database of extracted frequent routes, which are protected as private choices.

Given a user trajectory, we first reveal the stay regions and split a complex trajectory into several individual routes. Then, we map the subdivided routes into grid-based regions. To mine the frequent routes of a user, a series of routes that may occur at a similar time window is grouped together.

A frequency-correlated algorithm extracts frequent routes from each route group. We divide each route into directed

Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.

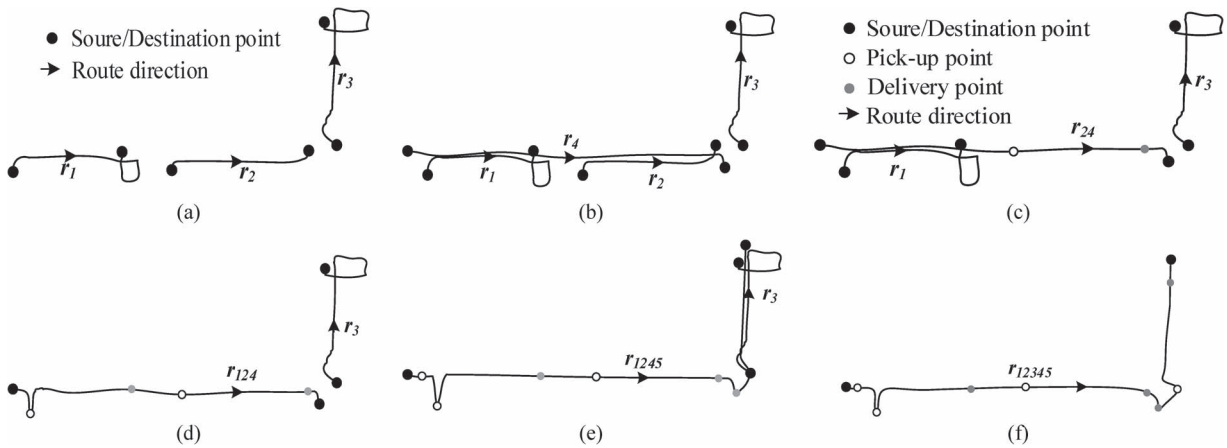


Fig. 2. Generation of a shared route by merging the frequent routes taken by five participating riders. (a) Start with three frequent routes (Alg.1). (b) New route r_4 similar to r_1 and r_2 (Alg.2). (c) Routes r_4 and r_2 merged into route r_{24} (Alg.3). (d) Route r_{124} is merged from r_{24} and r_1. (e) Route r_{1245} is merged from r_5 and r_{124} (Alg.3). (f) Final route r_{12345} is generated for five riders.

edges. These edges that are frequently visited by users are selected as qualified edges. Then, qualified routes are extracted based on the qualified edges and are used for extracting frequent routes. All of the frequent routes mined are sent to an online route database. Riders' roles in a carpool group are decided through transport mode recognition. If no proper rider is found, the route is stored in a database for matching in the future.

Our ultimate goal is to put together a group of ridesharing partners with an optimized route, which is based on riders' preferred routes. Qualified partners are selected and ranked by a service cost function, which compromised several user preferences. An equilibrium test is made to test whether a new rider will upset other riders who already joined the carpool.

Finally, a routing merging algorithm generates the final route to be shared. Three algorithms are used to mine the frequent routes by qualified riders, generate the shared route, and perform equilibrium testing to preserve the suboptimality when the carpooling group is updated with new riders replacing departing riders.

An example of the main processes is shown in Fig. 2. In Fig. 2(a) there are three independent frequent routes, r_1, r_2, and r_3, which are mined from the users' trajectory. The three routes are not suitable to match with each other. They are stored in the route table and are waiting for a match in the future.

In Fig. 2(b) a new route r_4 is mined from a user's trajectory log. Through transport mode recognition, we learned that r_4 is a driver with a car. According to the route line and timing property, the system finds two qualified routes r_1 and r_2 from the route table and recommends to r_4.

In Fig. 2(c) r_2 is selected as the most qualified rider for r_4. Therefore, r_4 and r_2 are merged into a shared route r_{24}. In Fig. 2(d), route r_1 is selected and passes the equilibrium test. This means that merging route r_1 will not negatively impact routes r_2 and r_4. Thus, we have the merged route, i.e., r_{124}.

After applying algorithms 2 and 3, a new rider r_5 with a car is merged with r_{124} [see Fig. 2(e)] and r_3 successively. In this process, rider r_4 becomes a passenger and is no longer driving. Finally, in Fig. 2(f), a shared route r_{12345} is generated among the five riders.

B. Route Splitting, Mapping, and Grouping

GPS trajectory logs correspond to individual routes. We find the starting and ending points of each user's route. The raw trajectories may only contain one route in a daily log. For frequent routes, we have to segment the data into subdivided routes to benefit the matching process among multiple riders. A rider may exceeding a certain time limit in a region before reaching the final destination. If the time gap between two adjacent GPS points is longer than a threshold, there are two routes before and after the time gap.

A rider staying in a region sufficiently long creates a *stay region*. This stay region may not be part of a route. We eliminate these stay regions from the original GPS trajectories. Different from the work of Zheng *et al.* [7], we focus on routes without stay regions. When deleting stay regions from trajectories, we end up with a time gap between two adjacent grid points. We use a time threshold to measure the time gap between two adjacent points and to split trajectories into individual routes.

To discretize the data, routes are mapped into equally distributed geographical grid points as $\{g_1, \ldots, g_k\}$, where g_k is the identifier of the kth grid point. A set of consecutive points mapped into the same grid was grouped together. Then a route can be represented by a linked list, i.e., $r\{g_1 \rightarrow g_2, \ldots, g_{k-1} \rightarrow g_k\}$. We denote the linked grid points by directed edge e. A route r is represented by a sequence of edges $r\{e_1, e_2, \ldots, e_{k-1}\}$.

Extracting frequent routes is very time consuming. A sliding time window of fixed duration $(t - \omega, t + \omega)$ is used to cluster routes by their occurrence time, where ω is the threshold used. All the routes that intersected with the time window at time t will be grouped together for further mining. We denote R as the set of all routes within the same time window.

IV. Trajectory Mining Algorithm

To qualify as a frequent route, most of its directed edges must be frequently visited. Let us first define the frequent routes and the frequent edges.

Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.

Definition 1 (Frequent Route): A frequent route is a complete route in which a rider frequently passed through in the similar time of a day. There are two key factors when determining a frequent route. The first is its high occurrence and the second is the similarity in the occurrence time. A frequent route is denoted by a list of directed edges as $\{e_i, \ldots, e_m\}$.

A qualified route is an instance of a frequent route. Each time that a rider passed a frequent route will generate a route record, and we call this route instance as a qualified route of the frequent route. A *qualified edge* is a directed edge that is visited by more than α routes, where α is a threshold given by a rider. In a route set R, we use f to denote the *visit frequency* of a directed edge. A qualified edge e must satisfy $f(e) > \alpha$.

Definition 2 (Frequent Edge): A frequent edge is a directed edge that is visited by more than β qualified routes, where β is a threshold given by a rider.

A qualified edge frequently appears in routes, whereas a frequent edge frequently appears in qualified routes. A frequent route must have a number of qualified routes. If there is a frequent route among a route set, each of its directed edges must have appeared in qualified routes several times. This definition of a frequent edge enables a frequent route with all frequent edges.

A. Trajectory Mining Steps

Step 1—Finding Qualified Edges: The feature of a qualified route is that most of its directed edges are also frequently visited by other qualified routes (just the frequent edge). We need to find a qualified route. We find the qualified edges from all the directed edges first. We calculate the visit frequency of each directed edge. All directed edges with $f(e) > \alpha$ are selected as qualified edges.

Step 2—Finding Qualified Routes: We use the qualified edges to find the potential qualified routes. We use route score S_r to qualify a given route r. The route score shows the proportion of qualified edges in a route. We define a membership function as

$$h(x, \theta) = \begin{cases} 1, & \text{if } x > \theta \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

where x is the member (a rider or a route) and θ is a threshold. The route score is defined by

$$S_r(r_i) = \sum_{e_j \in r_i} h\left[f(e_j), \alpha\right]/n \tag{2}$$

where n is the number of directed edges in route r_i, $f(e_j)$ is the visiting frequency of edge e_j, and α is the threshold to qualify a directed edge.

The route score reflects the frequency of using a given route. The higher the route score, the more edges of the route are frequently traversed. A route with a route score higher than a certain threshold γ is considered qualified.

Step 3—Finding Frequent Edges: We find the frequent edges based on their scores. An edge score is used to count how many qualified routes pass through a given edge. The edge score S_e of a given directed edge is defined as

$$S_e(e_i) = \sum_{r_j \in L(e_i)} h[S_r(r_j), \gamma] \tag{3}$$

where L is the list of routes that passed through a given directed edge S_r is the route score, γ is the threshold for deciding a qualified route, and h is the membership function defined in (1). All edges with $S_e > \beta$ are selected as frequent edges.

Step 4—Updating Qualified Route: To prune the disqualified routes, we use the frequent edges instead of the qualified edges to repeat Steps 2 and 3 until no change occurs in the qualified route set. Then, the frequent edges are fixed. The new route score S_r is calculated using membership function h as

$$S_r(r_i) = \sum_{e_j \in r_i} h\left[S_e(e_j), \beta\right]/n. \tag{4}$$

Theorem 1: The iterative process specified in Algorithm 1 converges in $d(d \leq l - m + 1)$ cycles, where l is the number of routes in a route set R and m is the number of qualified routes.

Proof: We use $Q_r(d)$ to denote the qualified route set after cycle d. After each cycle, the route with a route score lower than γ is deleted from Q_r. Thus, $Q_r(d) \subseteq Q_r(d-1)$. If in cycle d, $Q_r(d) = Q_r(d-1)$. The iteration is ended. Otherwise, if in cycle $k = d$, at least one route is deleted from $Q_r(d-1)$, and the iteration continues until no route is deleted or no route is left in Q_r. The convergence cycle is thus bounded by $l - m + 1$. **Q.E.D.**

The pseudocode for the algorithm of mining frequent routes is given in Algorithm 1. The start and ending times of each frequent route is averaged over all qualified routes.

Algorithm 1: Mining frequent routes from candidate riders to join the carpool

Input: A route set R, qualified edge threshold α, frequent edge threshold β, and qualified route threshold γ
Output: A set of frequent edges E forming the frequent routes.
Procedure:
1. Forall e_i that appeared in R, **do**
  If $f(e_i) > \alpha$, add e_i to the set of qualified edges Q_e
2. Forall r_i in R, **do** calculate route score $S_r(r_i)$ using (2)
  If $S_r(r_i) > \gamma$, add r_i to the set of qualified routes Q_r
3. $E = Q_e$ // Initialize the set of frequent edges
4. $C = |Q_r|$// The number of elements in Q_r
5. Forall e_i in E, **do**
  Calculate edge score $S_e(e_i)$ using (3)
  If $S_e(e_i) < \beta$, delete e_i from E
6. Forall r_i in Q_r, **do**
  Calculate route score $S_r(r_i)$ using (4)
  If $S_r(r_i) < \gamma$, delete r_i from Q_r //Updating Q_r
7. If $C < |Q_r|$, **go to** Step 4
8. Return with the set of E

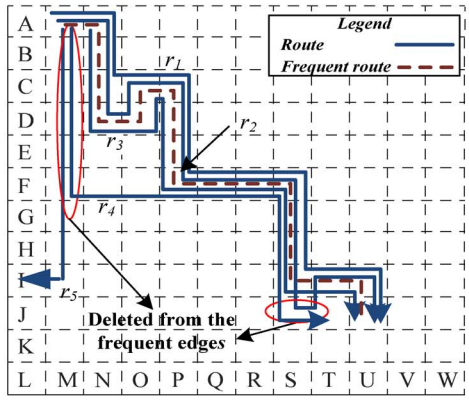
Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.

Finding qualified routes

Route	S_r	Route	S_r
r_1	0.94	r_2	0.95
r_3	1	r_4	0.81
r_5	0.55		

Finding frequent edges

Qualified edges	S_e
AM→AN	2
AM→BM	1
AN→BN	3
BM→CM	1
BN→CN	3
CM→DM	1
...	...



Updating qualified routes

Route	S_r	Route	S_r
r_1	0.94	r_2	0.86
r_3	1	r_4	0.38

Updating frequent edges

Frequent edges	S_e
AM→AN	2
AN→BN	3
BN→CN	3
CN→DN	2
CO→CP	3
...	...
JS→JT	1

Fig. 3. Trajectory mining to yield a frequent route. The tables record the qualified routes, the frequent edges, and their scores before and after route mining.

B. Mining Frequent Routes

Fig. 3 gives an example of mining the frequent route over five routes, i.e., $r_1, r_2, \ldots, r_5$. All qualified edges have been visited more than twice ($\alpha = 2$). We use the qualified edges to calculate the route score with (2). The result is given in the routes table, in the left-hand side. With a threshold $\gamma = 0.8$, there are four qualified routes, r_1, r_2, r_3, and r_4 (in the shaded entries), at the beginning. The edge score of each qualified edge is calculated using (3) and given in the edges table in the left-hand side.

In the first iteration, we eliminate some low-score edges from the qualified edges. We set threshold β to be 2; thus, all updated frequent edges (in the shaded entries) must have an edge score greater than 2. Edges AM $\rightarrow$ BM, BM $\rightarrow$ CM, $\ldots$, EM $\rightarrow$ FM are excluded. Finally, we end up with qualified routes r_1, r_2, and r_3 shown in the routing table in the right-hand side. Route r_4 is disqualified; thus, edge JS $\rightarrow$ JT is deleted.

C. Recognizing Transport Modes

The transport modes in users' GPS logs cover the use of buses, subways, bikes, walking, and cars. When accepting a rider, we consider his/her original transport mode. Only a rider with a car can be chosen as a driver in such a carpool group.

Initially, we examine the stop locations within a frequent route. A stop probability P measures the likelihood that a directed edge has a stop location. We calculate the passing speed v on each directed edge. If v is lower than a speed threshold φ, the rider likely stopped in this directed edge. The probability of a rider always stopping here is

$$l(x, \theta) = \begin{cases} 1, & \text{if } x \leq \theta \\ 0, & \text{otherwise} \end{cases}$$

$$P(e_j) = \sum_{e_j \in L_q(e_j)} l\left[v(e_j), \varphi\right] / S_e(e_j) \tag{5}$$

where L_q is the list of qualified routes that passed through a given directed edge and S_e is the edge score, which denotes the number of qualified routes that passed the given directed edge. Then, a frequent edge with a stop probability higher than a certain threshold τ is deemed as a stop location.

Definition 3 (Regular Stop Rate): The regular stop rate (RSR) S of a frequent route is the number of stop regions within a certain distance. We calculate the RSR of the ith route by

$$S(r_i) = \sum_{e_j \in r_i} h\left[P(e_j), \tau\right] / d_i \tag{6}$$

where d_i is the length of r_i and the equation of h is defined in (1). Then, the transport mode of a frequent route with an RSR higher than a threshold is recognized as self-driving.

V. Ranking of Qualified Riders

Various route sharing methods were proposed in the past. One approach is to fill up the available seats as much as possible [12]. In our method, we attempt to yield the riders' preferences to encourage more commuters to join.

A. Rider Preference Survey Results

To learn the riders' preferences on ridesharing, we have made an online survey with 12 questions. We received 324 responses. Among them, 120 persons drive to work and 119 take public transportation. Only 17% have experienced ridesharing. The majority (92%) prefer to join carpool services. The survey results show that 46% would like to join ridesharing on their frequent routes, whereas 34% would not mind joining in a longer route than their own.

Half of the riders do not care about the number of passengers on a trip. However, they hope that the trip will not start more than 10 min from their regular departure time. Most users would only like to carpool with friends or neighbors. More than 85% require the pickup/delivery locations on their preferred route. Only 9% of drivers accept a detour for other riders.

B. Building Grid-Structured Route Table

A grid-structured route table is suggested to record the qualified grid points in a set G, which are associated with a set of frequent routes F. Set F to record the IDs of all frequent routes that traversed through grid point g_i. Set G to record all the grid

Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.

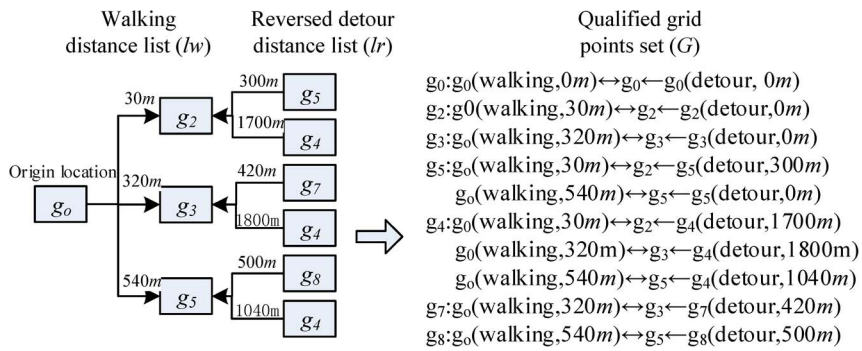


Fig. 4. Example of building up the set of qualified grid points set in the route table.

points that qualified the searching range of grid point g_i. Each record in G maintains a routing link. Different transport modes lead to four types of routing link, i.e.,

$$G_1 : g_k \{g_i(\text{walking}, d_{i,j}) \leftrightarrow g_j \leftarrow (\text{detour}, d_{k,j})g_k\}$$

$$G_2 : g_k \{g_i(\text{walking}, d_{i,j}) \leftrightarrow g_j \rightarrow (\text{detour}, d_{j,k})g_k\}$$

$$G_3 : g_k \{g_i(\text{detour}, d_{i,j}) \rightarrow g_j \leftrightarrow (\text{walking}, d_{k,j})g_k\}$$

$$G_4 : g_k \{g_i(\text{detour}, d_{j,i}) \leftarrow g_j \leftrightarrow (\text{walking}, d_{j,k})g_k\}.$$

The directed arrows indicate the movement direction between two grid points and d indicates the distance. Set G is a static set and can be prebuilt according to the road map. For simplicity we use three lists for each grid point g_i. The first list lw has all grid points within a walking distance. The second list ld lists the grid points within a detour distance to the point g_i. The third list lr has the grid points within a detour distance from point g_i. Then, we build set G by combining the three lists.

Fig. 4 presents an example of building set G_1 based on lists lw and lr. The directed arrows indicate the movement direction. Use g_o and g_4 as an example. There are three routes that connect points g_o and g_4, where g_2, g_3, and g_5 are the middle points. By checking the requirements of a minimal walking distance or a minimal detour distance, we could choose a desired routing path between g_0 and g_4.

C. Searching for Qualified Riders

Consider a route r_j; we try to find the fitting riders from the route table. According to the recognized transport mode, the rider can be set as a driver or a passenger and finds a proper share group. For a passenger, we need to find a driver or a shared route (there is a driver already). For a driver, we find a new passenger or another shared route (the rider becomes a passenger) according to the rider's will. We first find a group of candidate routes satisfying the following two specific qualifications.

1) The pickup and delivery locations are within a walking distance to the rider's home and destination location. All pickup and delivery points must be within a tolerable detour distance to the original shared route line.
2) The pickup and delivery time should be within a time window of the rider's home departure and arrival times. Suppose the pickup and delivery grid points of the rider are located at g_i and g_j; the driver needs to detour from grid points g_m and g_n separately, and then the time property should satisfy

$$t_d - t_w \leq t_m + t_{mi} \leq t_d + t_w \tag{7a}$$

$$t_a - t_w \leq t_n + t_{jn} \leq t_a + t_w \tag{7b}$$

where t_d and t_a are the original departure and arrival times of the rider, t_w is the time window, t_m and t_n are the driver's arrival time, and t_{mi} is the travel time between grid points g_m and g_i. Qualified riders are selected to meet the rider's preference. The preference factors are classified into the following five categories.

P1: Minimizing the total travel costs. The basic cost in a carpool ride is roughly proportional to the riding distance. For a typical rider p, suppose the individual route length is d_p, and D_m is the travel distance shared by m riders. Then, the *ride cost* of a rider is estimated by $\sum_{m=1}^{n}(D_m/m)/d_p$, where n is the total number of riders in the carpool.

P2: Minimizing the walking distance to make connections. This is quantified by $d_w/d_{\max}$, where d_w is the walking distance and $d_{\max}$ is the maximal walking distance accepted by a rider.

P3: Minimizing the detour distance to pick up other riders. This is measured by $(\sum_{m=1}^{n} D_m - d_p)/(\lambda \times d_p)$, where D_m is the travel distance shared by m riders; d_p is the route length; and λ is used to control the maximal detour distance acceptable, and its value is usually set between [0, 0.5].

P4: Reducing the advance time to start a carpool. This is measured by $\max(t_p - t_o, 0)/t_{\max}$, where t_p and t_o are the pick-up and original start times for a rider and $t_{\max}$ is the maximal accepted advanced time for the rider to start the travel.

P5: Minimizing the social distance. The social distance measures the relationship between riders, such as neighbors, colleagues, and friends. We simply measure the distance between the riders' home and office, i.e., $\min(d_s, d_d)/\zeta$, where d_s and d_d are the starting and destination distances between the rider and the driver, respectively. Threshold ζ is used to control the social distance.

We combine the preference factors into a *service cost* S_{cost}. All qualified riders are ranked according to their service costs. The pseudocode for searching and ranking the qualified riders is given in Algorithm 2.

Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Ranking qualified riders as candidates to join a carpool group

Input: A query route r
Output: A group of qualified riders Q
Procedure:
1. Forall grid points g_i in a route r, **do**
Initialize starting set $O = \{$all start grid points in $r\}$
Initialize destination set $D = \{$all destined points in $r\}$
2. For route r_i with a starting point in O in the same time window (7a), accept r_i in set R_s as a qualified rider
3. For any route r_j with a destined point in D in the same time window (7b), include r_j in the set of destination routes R_d
4. Form the set of routes $S \leftarrow R_s \cap R_d$ // Candidate riders
5. Forall routes r_i in S,
Calculate $S_{\text{cost}}(r_i)$ and $S_{\text{cost}}(r)$ using (9)
6. **If** $S_{\text{cost}}(r_i) < 1$ **and** $S_{\text{cost}}(r) < 1$,
Include r_i as a qualified rider in Q
7. Rank all r_i in Q by S_{cost}
8. Return the set of qualified riders Q

VI. Generating the Shared Route

We will define a service cost in Section VII when we get to the routing performance evaluation. Every time a new rider is recruited, the time plan and the service cost S_{cost} of existing riders may be affected. Therefore, a new rider must pass the following *equilibrium condition*.

The new time plan will agree with the time limitation for all existing riders in this route. The S_{cost} of all riders do not decrease more than a threshold, and S_{cost} are not higher than their initial cost. The *equilibrium condition* is defined by

$$S_{\text{cost_new}} < S_{\text{cost}} \qquad (S_{\text{cost_new}} - S_{\text{cost}})/S_{\text{cost}} < \rho \quad (8)$$

where $S_{\text{cost_new}}$ is the new service cost after adding a rider, S_{cost} is the old service cost, and ρ is a threshold to control the decrease in the service cost. The pseudocode for equilibrium testing and shared route generation is given in Algorithm 3.

Algorithm 3: Pairwise merging to generate the final route for the carpool group to share

Input: The driver's route r with a list of anchor points A, a list of qualified routes Q with pickup routing (g_o, g_p, and g_i), delivery routing (g_j, g_l, and g_d), and equilibrium test threshold ρ
Output: a shared route r by all accepted riders
Procedure:
1. If $Q \neq \phi$ gets the top route q_i from Q
Else store route r in the routing table
2. Insert detour points g_i and g_j into anchor list A
3. Select the next point g_a after g_i from list A
4. If $g_a == g_j$ // no other riders on route r being tested
Generate route from pickup point g_p to delivery point g_l

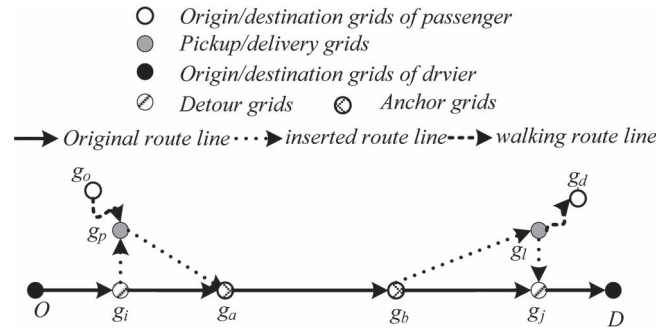


Fig. 5. Shared route based on merging similar frequent routes used by the qualified riders.

5. For all points g_n after g_j in A
If the driver cannot arrive at point g_n within the window of (7a), **Go to** Step 1
6. Insert g_p and g_l into list A at locations g_i and g_j
7. Else allow other riders to join route r
8. Generate a route from detour point g_p to anchor point g_a
9. For points g_n from g_a to g_{j-1} in list A
If the driver cannot arrive at point g_n within the time window of (7a), **Go to** Step 1
10. Select the previous point g_b before g_j from list A
11. Generate the route from anchor g_b to delivery point g_l
12. For all grid points g_n from g_{j+1} to D in list A
If the driver cannot arrive at point g_n within the time window of (7b), **Go to** Step 1
13. Use g_p and g_l in list A
14. For all riders, calculate $S_{\text{cost_new}}$ using (9)
15. **If** $S_{\text{cost_new}}$ does not satisfy the equilibrium test specified in (8), Go to Step 1
16. Return with the final route r

Fig. 5 shows how to merge a new rider into shared routes. The original route line is from $O \rightarrow g_a \rightarrow g_b \rightarrow D$, where g_a and g_b are the pickup and delivery grids (anchor grids could not be detoured) of a passenger, respectively. g_o and g_d are the source and destination grid points of a new selected passenger, respectively. The final merged route is indicated by $O \rightarrow g_p \rightarrow g_a \rightarrow g_b \rightarrow g_l \rightarrow D$. g_p and g_l are the new anchor grids searched from the routing table. The pickup routing is $\{g_o(\text{walking}) \rightarrow g_p \leftarrow g_i(\text{detour})\}$, and the delivery routing is $\{g_j(\text{detour}) \leftarrow g_l \rightarrow g_d(\text{walking})\}$. This process is recursively performed on a pairwise basis until all qualified riders are tested.

VII. Performance Analysis and Effects

We analyze the performance of the proposed routing method using the GeoLife data sets [7], [10]. To evaluate the effectiveness of the frequent route mining algorithm, we invite a group of volunteers to rank the mining results. We report the results in carpooling experiments. The GeoLife data set is collected from 182 users' GPS logs over four years from 2007. The data set contains 18 670 trajectory logs for an average of 103 trajectory logs per rider.

Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.

TABLE I
WEIGHT DISTRIBUTIONS OF THE FIVE PREFERENCE VECTORS

No.	Preference	Function	Default weight for passenger	Default weight for driver
1	Minimizing total travel cost	$\sum_{m=1}^{n}(D_m/m)/d_p$	w_1=0.5	w_1=0.5
2	Minimizing walking distance	$d_w/d_{\max}$	w_2=0.2	w_2=0
3	Minimizing detour distance	$(\sum_{m=1}^{n}D_m-d_p)/(\lambda\times d_p)$	w_3=0.1	w_3=0.3
4	Reducing the advance time	$Max(t_p-t_o,0)/t_{\max}$	w_4=0.1	w_4=0
5	Minimizing social distance	$Min(d_s,d_d)/\zeta$	w_5=0.1	w_5=0.2

This GeoLife data set can be found at http://research.microsoft.com/en-us/downloads/b16d359d-d164-469e-9fd4-daa38f2b2e13/default.aspx. We have mined more than 500 frequent routes from the data set. Finally, we recommend various groups of ridesharing partners. Table I specifies five rider-preferred vectors according to the five previously mentioned rules. This corresponds to a typical distribution.

A. Performance Analysis of Carpool Services

Given the rider preference weights, we proposed a service cost function for evaluating the quality of carpool services in the following. We use this function to conduct the performance evaluation experiments.

Theorem 2: The service cost of carpool is estimated by the following weighted summation of five component costs:

$$S_{\text{cost}} = w_1 \times \frac{\sum_{m=1}^{n}(D_m/m)}{d_p} + w_2 \times \frac{d_w}{d_{\max}} + w_3 \times \frac{2\times(\sum_{m=1}^{n}D_m - d_p)}{d_p} + w_4 \times \frac{\max(t_p - t_o,\,0)}{t_{\max}} + w_5\left(\frac{\min(d_s,\,d_d)}{\zeta}\right) \quad (9)$$

where w_i are the rider preference weights with $\sum_{i=1}^{5} w_i = 1$, $w_i > 0$.

Proof: The first item is the rate between the average driving distance with and without a carpool. The second item is the rate between the walking distance to attend a carpool and the maximal accepted walking distance by the rider. The third item is the rate of the extra driving distance (detour for other riders). The fourth item is the rate between the advanced starting time of the ride and the maximal accepted advanced time by the rider. In addition, the last item is the rate between the minimal origin and the destination distances between the passenger and the driver. **Q.E.D.**

The *success rate* is the fraction of queries that succeed in finding qualified riders to join the carpool group. Let U be the number of riders that successively finds a group of qualified riders. Let V be the total number of riders in the application pool. We define the success rate as

$$S = U/V. \quad (10)$$

This rate reflects the effectiveness of the ridesharing system. We report the measured success rate in our experiments.

TABLE II
STORAGE REQUIREMENTS OF OUR METHOD

	Original grid. points	After deleting stay points	After point grouping	After route extraction
Storage	23.7 M	16 M	1M	21.5 K
Reduction ratio	1	0.676	0.044	0.0009

The *mileage saving rate* (MSR) is the percentage of miles saved after adopting a ridesharing route compared with the total sum of individual mileage costs without a carpool service.

Theorem 3: The MSR of a carpool is estimated by

$$M = \left(\sum_{i=1}^{n} d_i - D_s\right) \Big/ \sum_{i=1}^{n} d_i \quad (11)$$

where n is the carpool size, d_i is the individual driving distance by the ith rider before joining the carpool, and D_s is the driving distance by the carpool group.

Proof: The numerator is the total mileage saved by the carpool group, and the denominator is the summation of all driving distances by adding up all individual riding distances before the carpool. The ratio indicates the mileage saving achieved. **Q.E.D.**

B. Mining Accuracy and Mining Time

To evaluate the effectiveness of the mining method, we performed an experiment to collect 37 riders' daily routes for one week against the route generated by the mining algorithm. The riders are asked to rank the mined route by a simple QoS criterion reflecting their satisfaction of the results.

The ranking is based on mining the frequent route against their preferred frequent routes in terms of the recall rate and the convenience degree. Each rider ranks the mined route in five levels of QoS [1], corresponding to the route being {poor, fair, good, very good, and excellent}, respectively.

We have mined 72 frequent routes from 37 riders. The average rating scores are: a recall rating of 4.1, a precision of 4.6, and a transport mode of 3.8 out of a perfect score of 5. These ratings imply that most riders are satisfied with the mining result. By analyzing the missing cases, we found that some riders may take the shared route most the time, but they may take a bus or a taxi due to special circumstances.

In Table II we give the storage requirement of the proposed carpool model. The first row is the number of records and the second row is the storage ratio between the numbers of the original data set. Obviously, the final grid-based route table is a tiny subset of the original data set. Assume 200 KB per each frequent route; the extracted route database only requires a total storage requirement of 21.5 K $\times$ 200 KB $=$ 4.30 GB.

The RSR is mostly indicated by the speed (in kilometers per hour) of the riding vehicle. In the GeoLife data set, there are 69 users who have labeled some of their trajectories with a transport mode. Among these routes, we have successfully mined 89 frequent routes in the experiments. We use these routes to evaluate the transport recognition process.

Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.

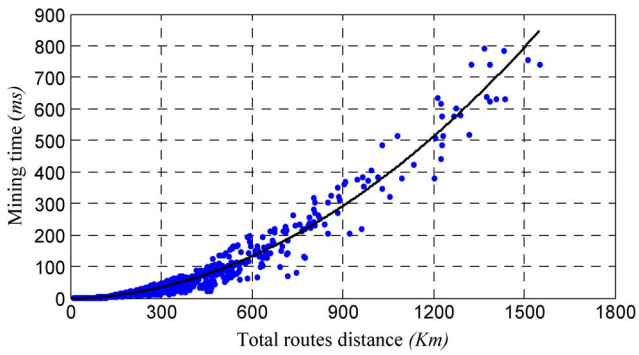


Fig. 6. Mining time measured in milliseconds plotted as a function of the routing distance in kilometers.

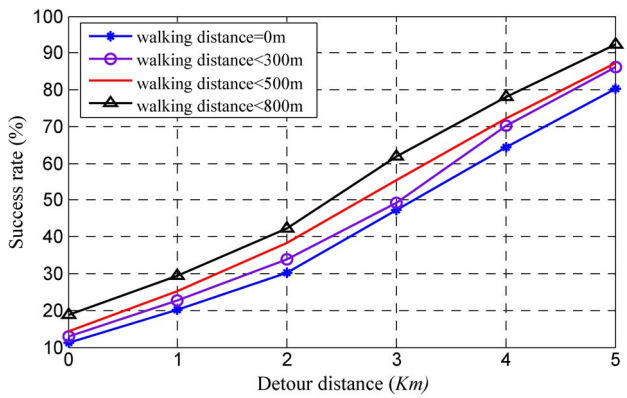


Fig. 7. Success rate in selecting ridesharers.

VIII. Measured Performance Results

We test our carpooling system on 500 riders from the GeoLife database on a monthly basis. We use simulation runs to verify the advantages and reveal limitations. The system can be easily implemented on a notebook computer. In this section, we report the experimental results, which verify the effectiveness of our carpooling routing scheme.

Fig. 6 plots the *frequent route mining time* against the total route distance among a group of routes. The mining time is less than 1 s, even when the total route distance reaches 1500 km. Driving in an urban area, the routing distance is usually less than 50 km; therefore, the mining time among a group of ten routes is less than 100 ms, which is rather small. This implies that our route mining process can be dynamically adjusted in real time.

Fig. 7 plots the success rate in selecting candidate riders for carpooling against the detour distance. From the plotted result, we see that an increasing walking and detour distance leads to a higher success rate. This is intuitively true because the shared route can be easier to form if the participant riders can tolerate a longer detour distance to make the carpool connections.

The longer the detour distance, the lower efficiency (average MSR) is achieved by a carpooling service. In Fig. 8, we see that, when the detour distance is larger than 4 km, the MSR could be lower than 5%. Thus, the detour distance should be maintained as short as possible. Excessive detouring will lose the carpool efficiency.

In practice, the rider detour distance is controlled to be less than 2 km. Thus, a success rate of 30%–40% is expected

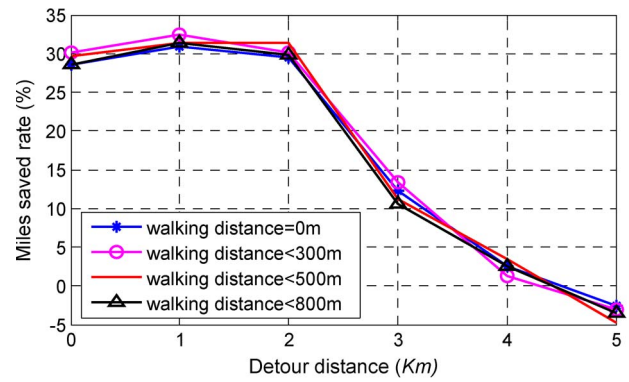


Fig. 8. Mileage saved against the detour distance.

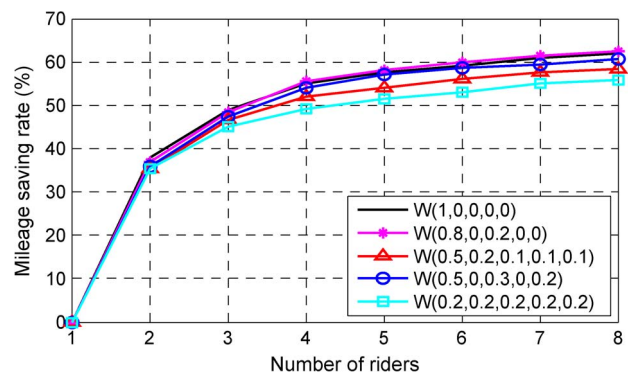


Fig. 9. Riding distance saved under five rider preference distributions.

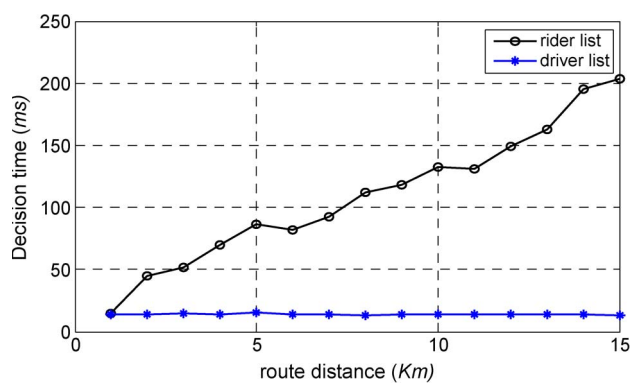


Fig. 10. Shared rider qualification time against the route distance.

with an MSR of 30%. Fig. 9 gives the final MSR under two rider preference weight distributions indicated by vector $W = (w_1, w_2, w_3, w_4, w_5)$. The distribution corresponds to the typical weight distributions in five rider preferences.

In Fig. 9 the larger the rider number (e.g., eight in a mini carpool van), the higher the mileage saving effect. The efficiency of the ridesharing increases with the carpool size. Preferences 1, 3, and 5 focus more on a lower riding cost, a shorter detour distance, and a closer social distance. Preferences 2 and 4 provide the riders with more convenience but less efficiency in the carpooling service. To maintain 50% mileage saving, we need at least four riders. With eight riders in the carpool the mileage saving can go up to 60%.

Fig. 10 gives the decision time of finding candidate drivers and passengers to join the carpool. The maximal walking distance is set to 800 m and the maximal detour distance is set

Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.

to 2 km. From the result, we could see that the response time of finding a candidate driver is unrelated to the total distance of the route. We only scan the qualified grid points around the origin and destination points. However, the decision time of searching qualified passengers is quite related to the route distance. That is because the longer the route, the more regions need to be searched.

IX. Conclusion and Suggestions

We have presented a new carpooling system that generates an efficient route for dynamically ridesharing by mining the frequent routes taken by participating riders. This GPS-assisted trajectory mining scheme is shown to be adaptive to dynamic urban traffic changes. The scheme is robust to traffic changes as the shared route is periodically updated in real time. The updating process takes rider preferences into account.

A. Summary of Contributions

Our contributions are summarized in four technical aspects.

1) We propose a frequency-correlated frequent route mining algorithm to infer a rider's frequent routes that could be easily distinguished from other routes.
2) We identify a new feature of the RSR to distinguish the transport mode between a public bus and private driving in a taxi, and private car driving.
3) We propose several carpool heuristics and strategies, which are based on a survey result from more than 300 participating riders divided into multiple groups.
4) We evaluate our new carpool service model using a large GPS data set from GeoLife. This data set contains 178 realistic user GPS trajectories over a period of four years. More than 30 users have participated in mining their regular routes using their personal trajectory logs.

B. Suggestions for Extended Work

The software for GPS trajectory mining and suboptimal carpool routing generation can be easily ported on a notebook computer. The route decision time is rather small (200 ms in Fig. 10). The accumulated database of the frequent routes by potential riders takes a small space (4.30 GB in Table II). The carpooling scheme can be applied in real time to serve the general public once the training database is built up. Two research extensions are suggested as follows.

1) Ridesharing recommendations from GPS trajectories are shown to be effective as a personalized service in urban traffic management. Our GPS trajectory mining of frequent routes can be used as a foundation to other personalized ridesharing services, such as long-distance travel on a shared vehicle.
2) In extended traveling services, the coverage scope is far beyond the city limit or the metropolitan area. Of course, some of the optimization parameters used in our carpooling model have to be modified. Another possible extension of this paper could be directed toward the combined use of private cars and public transportation.

References

[1] Z. Xiang, C. Chu, and H. Chen, "The study of a dynamic dial-a-ride problem under time dependent and stochastic environments," *Eur. J. Oper. Res.*, vol. 185, no. 2, pp. 534–551, Mar. 2008.

[2] W. He, D. Li, T. Zhang, L. An, M. Guo, and G. Chen, "Mining regular routes from GPS data for ridesharing recommendations," in *Proc. ACM Int. Workshop Urban Comput.*, Beijing, China, Aug. 2012.

[3] K. Ghoseiri, A. Haghani, and M. Hamedi, "Real-time rideshare matching problem," U.S. Dept. of Transp., Univ. of Maryland, College Park, MD, USA, Tech. Rep. UMD-2009-05, Jan 2011.

[4] K. Chang, L. Wei, M. Yeh, and W. Peng, "Discovering personalized routes from trajectories," in *Proc. ACM Int. Workshop Location-Based Social Netw.*, Chicago, IL, USA, 2011, pp. 33–40.

[5] L. Chen, M. Lv, Q. Ye, G. Chen, and J. Woodward, "A personal route prediction system based on trajectory data mining," *Inf. Sci.*, vol. 181, no. 7, pp. 1264–1284, Apr. 2011.

[6] N. Agatz, A. Erera, M. Savelsbergh, and X. Wang, "Dynamic Ride-Sharing: A Simulation Study in Metro Atlanta," in *Proc. Int. Symp. Transp. Traffic Theory*, 2011, vol. 17, pp. 532–550.

[7] Y. Zheng, Q. Li, Y. Chen, X. Xie, and W. Ma, "Understanding mobility based on GPS data," in *Proc. Conf. Ubiquitous Comput.*, Seoul, Korea, 2008, pp. 312–321.

[8] C. Cho, Y. Wu, C. Yen, and C. Chang, "Passenger Search by Spatial Index for Ridesharing," in *Proc. Int. Conf. Technol. Appl. Artif. Intell.*, Nov. 2011, pp. 88–93.

[9] J. P. Boavida, V. Kamat, D. Nakum, R. Nong, C. W. Wu, and X. Zhang, "Algorithms for the carpool problem," *IMA Preprint Series, 2133-6*, Aug. 2006.

[10] Y. Zheng, X. Xie, and W. Ma, "GeoLife: A collaborative social networking service among user, location and trajectory," *IEEE Data Eng. Bulletin*, vol. 33, no. 2, pp. 32–40, 2010.

[11] R. Baldacci, V. Maniezzo, and A. Mingozzi, "An exact method for the car pooling problem based on Lagrangean column generation," *Oper. Res., INFORMS*, vol. 52, no. 3, pp. 422–439, Jun. 2004.

[12] G. Berbeglia, J.-F. Cordeau, and G. Laporte, "Dynamic pickup and delivery problems," *Eur. J. Oper. Res.*, vol. 202, no. 1, pp. 8–15, Apr. 2010.

[13] N. Agatz, A. Erera, M. Savelsbergh, and X. Wang, "Optimization for dynamic ride-sharing: A review," *Eur. J. Oper. Res.*, vol. 223, no. 2, pp. 295–303, Dec. 2012.

[14] R. Calvo, F. de Luigi, P. Haastrup, and V. Maniezzo, "A distributed geographic information system for the daily carpooling problem," in *Computer Operation Research*. Amsterdam, The Netherlands: Elsevier, 2004.

[15] E. Kamar and E. Horvitz, "Collaboration and shared plans in the open world: Studies of ridesharing," in *Proc. 21st Int. Joint Conf. Artif. Intell.*, Pasadena, CA, USA, 2009, pp. 187–194.

[16] S. Yan, C. Chen, and Y. Lin, "A model with a heuristic algorithm for solving many-to-many car pooling problem," *IEEE Trans. Intell. Transp. Syst.*, vol. 12, no. 4, pp. 1362–1373, Dec. 2011.

[17] R. Trasarti, F. Pinelli, M. Nanni, and F. Giannotti, "Mining mobility user profiles for car pooling," in *Proc. ACM Int. Conf. Knowl. Discovery Data Mining*, 2011, pp. 1190–1198.

[18] B. J. Coltin and M. Veloso, "Towards ridesharing with passenger transfers," in *Proc. Int. Conf. Auton. Agents Multi-Agent Syst.*, 2013, pp. 1299–1300.

[19] F. Giannotti, M. Nanni, F. Pinelli, and D. Pedreschi, "Trajectory pattern mining," in *Proc. ACM Int. Conf. Knowl. Discovery Data Mining*, San Jose, CA, USA, 2007, pp. 330–339.

[20] J. Lee, J. Han, X. Li, and H. Cheng, "Mining discriminative patterns for classifying trajectories on road networks," *IEEE Trans. Knowl. Data Eng.*, vol. 23, no. 5, pp. 713–726, May 2011.

[21] G. Roh, J. Roh, S. Hwang, and B. Yi, "Supporting pattern-matching queries over trajectories on road networks," *IEEE Trans. Knowl. Data Eng.*, vol. 23, no. 11, pp. 1753–1758, Nov. 2011.

[22] S. Liu, Y. Yue, and R. Krishan, "Adaptive collective routing using Gaussian process dynamic congestion model," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2013, pp. 704–712.

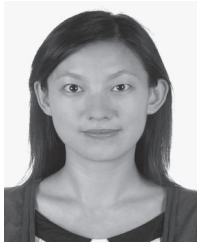
[23] S. Liu, Y. Liu, L. Ni, J. Fan, and M. L. Li, "Towards mobility–based clustering," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovering Data Mining*, 2010, pp. 919–928.

[24] N. Diffield and M. Grossglauser, "Trajectory sampling for direct traffic observation," in *Proc. SIGCOMM*, 2000, pp. 271–282.

[25] X. Li, D. Tang, and X. Xu, "Driving features of traffic flow around an intersection from trajectory of vehicles," in *Proc. Geoinformat.*, 2010, pp. 1–5.

[26] S. Shang, R. Ding, B. Yuan, K. Xie, K. Zheng, and P. Kainis, "User-oriented trajectory search for trip recommendation," in *Proc. EDBT*, 2012, pp. 156–167.

Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.



Wen He is working toward the Ph.D. degree in computer science and technology in the Department of Computer Science and Technology, School of Information Science and Technology, Tsinghua University, Beijing, China.

Her research interests include intelligent transportation systems, data mining, and location-based social networks.

Kai Hwang (LF'13) received the Ph.D. degree from University of California Berkeley, Berkeley, CA, USA.

He is a Professor of the Department of Electrical Engineering, University of Southern California, Los Angeles, CA, USA. He is also an EMC-endowed Visiting Chair Professor of the Department of Computer Science and Technology, Tsinghua University, Beijing, China. He has published extensively in computer architecture, parallel processing, network security, distributed systems, and cloud computing.

Deyi Li received the Ph.D. degree in computer science from Heriot–Watt University, Edinburgh, U.K.

He is a Professor of software engineering with the School of Software Engineering, Tsinghua University, Beijing, China. His research interests include networked data mining, artificial intelligence with uncertainty, and cloud computing.

Dr. Li is a member of the Chinese Academy of Engineering.

Authorized licensed use limited to: San Jose State University. Downloaded on April 06,2026 at 05:24:03 UTC from IEEE Xplore. Restrictions apply.